

代码即策略：用于具身控制的语言模型程序

Jacky Liang, Wenlong Huang, Fei Xia, Peng Xu, Karol Hausman, Brian Ichter, Pete Florence, Andy Zeng

Robotics at Google

摘要——在大规模代码补全任务上训练的大语言模型（Large Language Models）已被证明能够根据文档字符串合成简单的 Python 程序 [1]。我们发现，这些具备代码编写能力的大语言模型可以被重新用于编写机器人策略代码，只需给定自然语言命令。具体而言，策略代码可以表达函数或反馈回路，用于处理感知输出（例如来自目标检测器 [2]、[3] 的输出），并对控制原语应用程序编程接口进行参数化。当以少量示例提示的方式提供若干示例语言命令（以注释形式呈现）及其对应的策略代码时，大语言模型能够接收新命令，并自主重新组合应用程序编程接口调用，以分别生成新的策略代码。通过串联经典逻辑结构并引用第三方库（如 NumPy、Shapely）来执行算术运算，以这种方式使用的大语言模型能够编写出具备以下能力的机器人策略：(i) 展现空间几何推理能力，

(ii) 泛化到新指令，以及 (iii) 根据上下文（即行为常识）为模糊描述（如“更快”）指定精确值（如速度）。本文提出代码即策略：一种以机器人中心的语言模型生成程序（Language Model Generated Programs）表述，能够表示反应式策略（如阻抗控制器）以及基于航点的策略（基于视觉的抓取与放置、基于轨迹的控制），并在多个真实机器人平台上进行了演示。我们方法的核心是提示分层代码生成（递归定义未定义的函数），这能够编写更复杂的代码，并将最先进的水平提升至在 HumanEval[1] 基准上解决 39.8% 的问题。

代码和视频可在 <https://code-as-policies.github.io> 获取

I. 引言

使用语言的机器人需要语言被接地（或情境化），以指涉物理世界并在词语、感知和动作之间建立联系 [4]。经典方法通过词汇分析来接地语言，提取语义表示以指导策略 [5]–[7]，但往往难以处理未见过的指令。较新的方法端到端地学习接地（从语言到动作）[8]–[10]，但需要大量训练数据，而在真实机器人上获取这些数据成本高昂。与此同时，自然语言处理的最新进展表明，在互联网规模数据上预训练的大语言模型（LLMs）[11]–[13] 展现出开箱即用的能力 [14]–[16]，可应用于使用语言的机器人，例如从自然语言指令规划一系列步骤 [16]–[18]，无需额外的模型微调。这些步骤可以通过固定技能集（即通过行为克隆或强化学习预训练的策略）中的价值函数接地到真实机器人的可供性 [19]–[21]。尽管这一方法有前景，但这种抽象阻止了大语言模型直接影响感知-动作反馈回路，使得以下方式接地语言变得困难：

(i) 泛化共享感知和动作的反馈模式，例如从“把苹果放在橙子上”到“看到橙子时把苹果放下”，(ii) 在控制中表达常识经验，例如“移动更快”“推得更用力”，或 (iii) 理解空间关系“把苹果向左移动一点”。因此，纳入每项新技能（以及接地模式）都需要额外的数据和重新训练——数据负担依然存在，只是转移到了技能获取上。这促使我们提出：如何

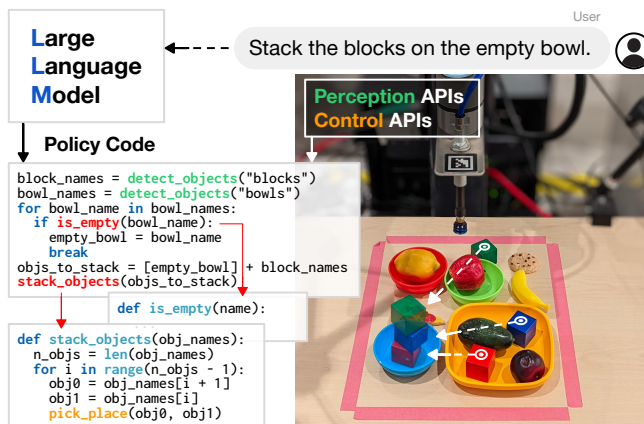


图 1：通过少样本提示给出的示例，机器人可以利用编写代码的大语言模型（LLMs）将自然语言命令翻译为机器人策略代码，该代码处理感知输出、参数化控制原语、为未定义函数递归生成代码，并泛化到新任务。大语言模型能否超越仅规划技能序列的范畴？本文发现，编写代码的大语言模型 [1]、[11]、[22] 能够更进一步：协调规划、策略逻辑与控制。基于代码补全训练的大语言模型已展示出从文档字符串合成 Python 程序的能力。我们发现，这些模型可被重新用于编写机器人策略代码，只需提供自然语言命令（以注释形式呈现）。策略代码可以表达处理感知输出（例如开放词汇目标检测器 [2]、[3]）的函数或反馈回路，并参数化控制原语应用程序编程接口（参见图 1）。当提供若干示例语言命令及对应的策略代码（通过少样本提示，以灰色显示）时，大语言模型可以接收新命令（以绿色显示），并自主重组应用程序编程接口调用以生成新的策略代码（高亮显示）：

```
# if you see an orange, move backwards.
if detect_object("orange"):
    robot.set_velocity(x=-0.1, y=0, z=0)
# move rightwards until you see the apple.
while not detect_object("apple"):
    robot.set_velocity(x=0, y=0.1, z=0)
```

编写代码的模型能够表达多种算术运算以及基于语言的反馈回路。它们不仅能泛化到新指令，而且由于在数十亿行代码和注释上训练，还能根据上下文为模糊描述（“更快”和“向左”）指定精确值（例如速度）——从而引出行为常识：

```
# do it again but faster, to the left, and with a banana.
while not detect_object("banana"):
    robot.set_velocity(x=0, y=-0.2, z=0)
```

将代码表示为策略继承了大语言模型的诸多优势：不仅具备解释自然语言的能力，还能通过将“say(text)”作为可用的动作原语应用程序编程接口，实现人机对话和问答：

```
# tell me why you stopped moving.
robot.say("I stopped moving because I saw a banana.")
```

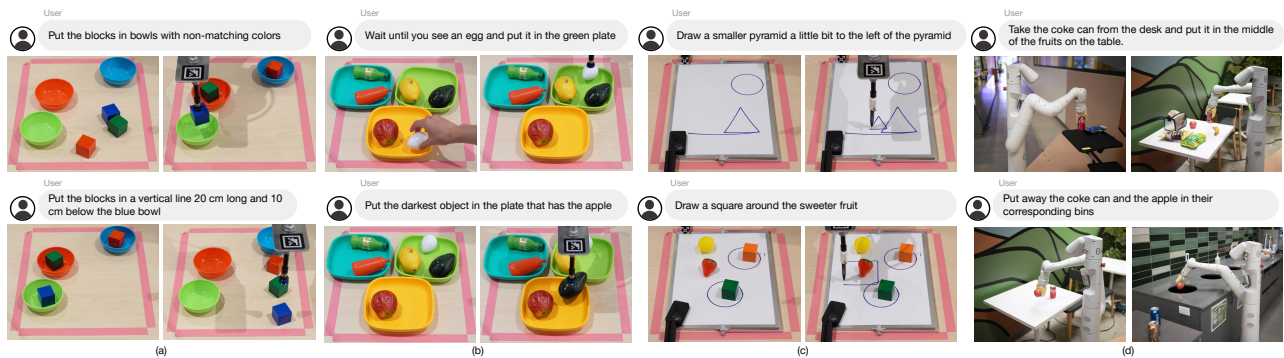


图 2：代码即策略能够跨不同领域和机器人遵循自然语言指令：桌面操作（a）-（b）、二维形状绘制（c），以及使用来自日常机器人（Everyday Robots）的机器人在厨房中进行移动操作（d）。我们的方法使机器人能够执行空间几何推理、解析物体关系，并使用现成模型和少样本提示形成多步行为，无需额外训练。完整视频和更多任务请访问 code-as-policies.github.io。我们提出代码即策略（Code as Policies, CaP）：一种以机器人中心的语言模型生成程序（LMPs）在真实系统上执行的表述。Python 风格的 LMP 可以表达复杂策略，使用：

- 经典逻辑结构，例如序列、选择（if/else）和循环（for/while），以在运行时组装新行为。
- 第三方库用于插值点（NumPy）、分析和生成形状（Shapely）以进行空间几何推理等。

LMP 可以是分层的：通过提示递归定义新函数，随时间积累自己的库，并自架构一个动态代码库。我们在多个机器人系统上证明，LLM 可以自主解释语言命令以生成 LMP，这些 LMP 表示反应式低级策略（例如 PD 或阻抗控制器）和基于航点的策略（例如用于基于视觉的拾取和放置，或基于轨迹的控制）。我们的主要贡献是：（i）代码即策略：一种使用 LLM 编写机器人代码的表述，（ii）一种分层代码生成方法，在机器人学和标准代码生成问题上均改进了最先进水平，在 HumanEval 上达到 39.8% 的 P@1 [1]，（iii）一个新的基准，用于评估未来语言模型在机器人代码生成问题上的表现，以及（iv）消融实验，分析 CaP 如何改进泛化度量 [23]，并且它遵循缩放定律——更大的模型表现更好。代码即策略提出了一种连接词语、感知和动作的新方法；在人类机器人交互中实现应用，但并非没有局限性。我们在第 V 节中讨论这些。完整的提示和生成的输出在附录中，可与其他结果、视频和代码一起在 code-as-policies.github.io 找到。

II. 相关工作

通过语言控制机器人 有着悠久的历史，包括通过自然语言的词汇解析实现人机交互的早期演示 [5]。语言不仅作为非专家与机器人交互的接口 [24]、[25]，还作为一种手段，以组合方式将泛化扩展到新任务 [9]、[17]。相关文献浩如烟海（我们参考 Tellex 等人 [4] 和 Luketina 等人 [26] 的综述以获取全面概述），但近期工作大致可分为高层解释（例如语义解析 [25]、[27]–[32]）、规划 [14]、[17]、[18] 以及底层策略（例如基于模型的 [33]–[35]、模仿学习 [8]、[9]、[36]、[37] 或强化学习 [38]–[42]）。相比之下，本文聚焦于大语言模型的代码生成方面，并将生成的程序用作控制机器人的一种表达方式。

大语言模型 展现出令人印象深刻的零样本推理能力：从规划 [14] 到编写数学程序 [43]；从解决科学问题 [44] 到使用训练好的验证器 [45] 处理数学应用题。这些能力可以通过提示方法进一步提升，例如从最少到最多 [46]、逐步思考 [15] 或思维链 [47]。与本文最密切相关的是那些在不进行额外模型训练的情况下，将大语言模型能力用于机器人智能体的工作。例如，Huang 等人通过文本补全和语义翻译将自然语言命令分解为可执行动作序列 [14]，而 SayCan[17] 通过联合解码由价值函数中的技能可供性 [20] 加权的大语言模型，为机器人生成可行的计划。内心独白 [18] 通过纳入成功检测器或其他视觉语言模型的输出，并利用其反馈重新规划，扩展了大语言模型规划。苏格拉底模型 [16] 使用视觉语言模型将感知信息（以青色表示）替换到生成计划的语言提示中，并使用语言条件策略，例如用于抓取 [36]。以下示例说明了本文方法与上述先前工作之间的定性差异。当任务为“将可乐罐稍微向右移动”时：

LLM Plan [14], [17], [18]

```
1. Pick up coke can
2. Move a bit right
3. Place coke can
```

Socratic Models Plan [16]

```
objects = [coke can]
1. robot.grasp(coke can) open vocab
2. robot.place_a_bit_right()
```

先前工作生成的计划假设存在一种技能，使机器人能够将物体稍微向右移动。本文方法的不同之处在于，它使用大语言模型直接生成策略代码（其中嵌套了计划）以在机器人上运行，从而避免了为计划中的每一步预定义策略的要求：

Code as Policies (ours)

```
while not obj_in_gripper("coke can"):
    robot.move_gripper_to("coke can")
robot.close_gripper()
pos = robot.gripper.position
robot.move_gripper(pos.x, pos.y+0.1, pos.z)
robot.open_gripper()
```

Our approach (CaP) not only leverages logic structures to specify feedback loops, but it also parameterizes (and write parts of) low-level control primitives. CaP alleviates the need to collect data and train a fixed set of predefined skills or language-conditioned policies – which are expensive and often remain domain-specific.

Code generation has been explored with LLMs [1], [48] and without [49]. Program synthesis has been demonstrated to be capable of drawing simple figures [50] and generating policies

解决二维任务 [51]。本文在这些工作的基础上进一步拓展，证明：（一）编写代码的大语言模型无需先前工作所需的额外训练，即可实现新颖的推理能力（例如，借助第三方库的熟悉度编码空间关系）[35]、[36]、[52] – [56]；（二）分层代码编写（受递归摘要启发 [57]）提升了最先进的代码生成水平。本文还提出了一个以机器人技术为主题的新一代码生成基准，用于评估未来语言模型在机器人领域的表现。

三、方法

本节刻画了预训练大语言模型在何种程度上能够被提示生成作为策略的代码——表示为一系列语言模型程序。广义上，本文使用语言模型程序这一术语指代任何由语言模型生成并在系统上执行的程序。本研究探讨代码即策略，这是一类将语言指令映射为代码片段的语言模型程序，这些代码片段（一）对感知输入做出反应（即来自传感器或传感器上层模块的输入），（二）参数化控制原语应用程序编程接口，以及

（三）直接在机器人上编译并执行，例如：

```
# stack the blocks in the empty bowl.
empty_bowl_name = parse_obj('empty bowl')
block_names = parse_obj('blocks')
obj_names = [empty_bowl_name] + block_names
stack_objs_in_order(obj_names=obj_names)
```

输入指令被格式化为注释（绿色），可由人类提供或由另一个语言模型程序编写。大语言模型的预测输出（高亮显示）应为有效的 Python 代码，以自回归方式生成 [11]、[12]。语言模型程序通过少样本提示示例来生成不同的子程序，这些子程序可以处理目标检测结果、构建轨迹或对控制原语进行排序。语言模型程序可以通过组合已知函数（例如，使用感知模块的 `get_obj_names()`）或调用其他语言模型程序来定义未定义的函数，从而分层生成。

```
# define function stack_objs_in_order(obj_names).
def stack_objs_in_order(obj_names):
    for i in range(len(obj_names) - 1):
        put_first_on_second(obj_names[i + 1], obj_names[i])
```

其中 `put_first_on_second` 是现有的开放词汇拾取与放置原语（例如 CLIPort [36]）。对于新的机器人形态，这些主动函数调用可以替换为表示智能体动作空间的可用控制应用程序编程接口（例如 `set_velocity`）。通过函数式编程，带有详细变量名的分层代码生成可视为思维链提示 [47] 的一种变体。由语言模型程序（LMP）定义的函数可以随时间逐步累积，新的 LMP 可以引用先前构建的函数来扩展策略逻辑。执行 LMP 时，首先检查其安全性，确保没有导入语句、以 `_` 开头的特殊变量，或对 `exec` 和 `eval` 的调用。然后，调用 Python 的 `exec` 函数，将代码作为输入字符串，并提供两个字典作为代码执行的作用域：（i）`globals`，包含生成代码可能调用的所有应用程序编程接口；（ii）`locals`，一个空字典，将在 `exec` 执行期间填充变量和新定义的函数。如果 LMP 预期返回值，则在 `exec` 完成后从 `locals` 中获取。

A. 提示语言模型程序

生成 LMP 的提示包含两个要素：1. 提示信息，例如告知大语言模型（LLM）可用应用程序编程接口的导入语句，以及如何使用这些接口的类型提示。

```
import numpy as np
from utils import get_obj_names, put_first_on_second
```

2. 示例 是指令到代码的配对，以少样本“演示”的形式展示自然语言指令应如何转换为代码。这些示例可能包括执行算术运算、调用其他应用程序编程接口以及编程语言的其他特性。指令以注释形式直接写在相应解决方案代码块之前。我们可以通过将新指令和响应逐步追加到提示中来维护 LMP “会话”，使后续指令能够引用先前的指令，例如“撤销上一个操作”。

B. 示例语言模型程序（低级）

语言模型程序（LMP）或许通过示例最容易理解，以下部分从简单的纯 Python 指令逐步构建到能够完成机器人任务的更复杂指令。除非另有说明，本文中的所有示例和实验均使用 OpenAI Codex code-davinci-002，温度设为 0（即确定性贪心令牌解码）。此处，提示（灰色部分）以提示开头，表明我们正在编写 Python。随后给出一个示例，指定返回值的格式，并将其赋值给名为 `ret_val` 的变量。输入指令为绿色，生成的输出高亮显示：

```
# Python script
# get the variable a.
ret_val = a
# find the sum of variables a and b.
ret_val = a + b
# see if any number is divisible by 3 in a list called xs.
ret_val = any(x % 3 == 0 for x in xs)
```

第三方库。 编写 Python 代码的语言模型存储了许多流行库的知识。可以提示语言模型程序使用这些库来执行复杂指令，而无需编写所有代码，例如使用 NumPy 来引发坐标空间推理。此处的提示包括导入语句，示例定义了基本方向。变量名也很重要，以表明 `pts_np` 和 `pt_np` 是 NumPy 数组。对二维向量的操作意味着这些点也是二维的。示例：

```
import numpy as np
# move all points in pts_np toward the right.
ret_val = pts_np + [0.3, 0]
# move a pt_np toward the top.
ret_val = pt_np + [0, 0.3]
# get the left most point in pts_np.
ret_val = pts_np[np.argmin(pts_np[:, 0]), :]
# get the center of pts_np.
ret_val = np.mean(pts_np, axis=0)
# the closest point in pts_np to pt_np.
ret_val = pts_np[np.argmin(np.sum((pts_np - pt_np)**2, axis=1))]
```

第一方库。 语言模型程序还可以使用训练数据中不存在的第一方库（感知或控制原语 API），只要这些函数具有有意义的名称并在提示/示例中提供。例如（完整提示见 B.2）：

```

from utils import get_pos, put_first_on_second
...
# move the purple bowl toward the left.
target_pos = get_pos('purple bowl') + [-0.3, 0]
put_first_on_second('purple bowl', target_pos)
objs = ['blue bowl', 'red block', 'red bowl', 'blue block']
# move the red block a bit to the right.
target_pos = get_pos('red block') + [0.1, 0]
put_first_on_second('red block', target_pos)
# put the blue block on the bowl with the same color.
put_first_on_second('blue block', 'blue bowl')

```

提示为机器人领域导入了两个函数：一个通过名称获取物体的二维位置（使用开放词汇物体检测器 [2]），另一个将第一个物体放置在第二个目标上，目标可以是物体名称或二维位置。注意语言模型程序适应新指令的能力——第一个通过使用“一点”来修改移动幅度，而第二个将物体与“相同颜色”关联起来。语言推理可以通过编写代码的语言模型进行少样本提示（完整提示见 B.1），例如将物体名称与自然语言描述（“海色块”）、类别（“碗”）或过去上下文（“另一个块”）关联起来：

```

objs = ['blue bowl', 'red block', 'red bowl', 'blue block']
# the bowls.
ret_val = ['blue bowl', 'red bowl']
# sea-colored block.
ret_val = 'blue block'
# the other block.
ret_val = 'red block'

```

C. 示例语言模型程序（高级）

控制流。编程语言允许使用诸如条件分支（if-else）和循环语句等控制结构。此前我们展示了语言模型程序（LMPs）能够以列表推导式的形式表达 for 循环。此处我们展示它们如何编写 while 循环以构成一个简单的反馈策略。注意，该提示（与 B.2 中的相同）不包含此类示例：

```

# while the red block is to the left of the blue bowl, move it to the
right 5cm at a time.
while get_pos('red block')[0] < get_pos('blue bowl')[0]:
    target_pos = get_pos('red block') + [0.05, 0]
    put_first_on_second('red block', target_pos)

```

LMPs 可以被组合 通过嵌套函数调用实现。这使得能够在单个提示中包含更多少样本示例，以提高功能准确性和覆盖范围，同时保持在大型语言模型的最大输入令牌长度之内。以下内容（完整提示见 B.4）生成一个响应，该响应使用 parse_obj, 另一个语言模型程序，将对象名称与语言描述关联起来：

```

objs = ['red block', 'blue bowl', 'blue block', 'red bowl']
# while the left most block is the red block, move it toward the right.
block_name = parse_obj('the left most block')
while block_name == 'red block':
    target_pos = get_pos(block_name) + [0.3, 0]
    put_first_on_second(block_name, target_pos)
    block_name = parse_obj('the left most block')

```

The `parse_obj` LMP (full prompt in Appendix B.5):

```

objs = ['red block', 'blue bowl', 'blue block', 'red bowl']
# the left most block.
block_names = ['red block', 'blue block']
block_positions = np.array([get_pos(name) for name in block_names])
left_block_name = block_names[np.argmin(block_positions[:, 0])]
ret_val = left_block_name

```

LMP 可以分层生成函数以供未来复用：

```

import numpy as np
from utils import get_obj_bbox_xyxy
# define function: total = get_total(xs).
def get_total(xs):
    return np.sum(xs)
# define function: get_objs_bigger_than_area_th(obj_names, bbox_area_th).
def get_objs_bigger_than_area_th(obj_names, bbox_area_th):
    return [name for name in obj_names
            if get_obj_bbox_area(name) > bbox_area_th]

```

函数生成可以通过解析由语言模型程序（LMP）生成的代码、定位尚未定义的函数，并调用另一个专门用于函数生成的语言模型程序来创建这些函数，从而实现。这使得提示和由语言模型程序生成的代码都可以调用尚未定义的函数。提示工程师不再需要在示例中提供所有实现细节——代码逻辑的“粗略草图”可能就足够了。高层语言模型程序还可以遵循良好的抽象实践，避免将所有代码逻辑“扁平化”到单一层次。除了使生成的代码更易于阅读之外，这还提高了代码生成性能，如第 IV-A 节所示。定位尚未定义的函数也在生成函数的主体内部完成。注意，在上面的示例中，`get_obj_bbox_area` 不是提供的应用程序编程接口调用。相反，它可以根据需要生成：

```

# define function: get_obj_bbox_area(obj_name).
def get_obj_bbox_area(obj_name):
    x1, y1, x2, y2 = get_obj_bbox_xyxy(obj_name)
    return (x2 - x1) * (y2 - y1)

```

请注意，提示并未明确说明 `get_obj_bbox_xyxy` 返回什么，但名称暗示它包含轴对齐边界框的最小和最大 xy 坐标，而 LLM 能够推断出这一点并生成正确的代码。在 Python 中，我们通过解析代码块的抽象语法树并检查给定作用域中不存在的函数来实现分层函数生成。我们使用函数生成 LMP 来编写这些未定义的函数，并将它们添加到作用域中。此过程在生成的函数体上重复进行，以深度优先的方式分层创建新函数。结合控制流、LMP 组合和分层

函数生成。以下示例展示了语言模型程序（LMPs）如何结合这些能力来遵循更复杂的指令，并在桌面操作领域执行任务。为简洁起见省略了提示，但它们与之前的类似。高层语言模型程序生成高层策略行为，并依赖 `parse_obj` 通过语言描述获取对象名称：

```

objs = ['red block', 'blue bowl', 'blue block', 'red bowl']
# while there are blocks with area bigger than 0.2 that are left of the
red bowl, move them toward the right.
block_names = parse_obj('blocks with area bigger than 0.2 that are
left of the red bowl')
while len(block_names) > 0:
    for block_name in block_names:
        target_pos = get_pos(block_name) + np.array([0.1, 0])
        put_first_on_second(block_name, target_pos)
    block_names = parse_obj('blocks with area bigger than 0.2 that are
left of the red bowl')

```

然后，`parse_obj` 使用 `get_objs_bigger_than_area_th`（尚未定义的函数）来完成查询。该函数通过 `parse_obj` 提示的提示部分中的导入语句给出，但尚未实现。分层函数生成随后会如上所示创建此函数。

```

objs = ['red block', 'blue bowl', 'blue block', 'red bowl']
# blocks with area bigger than 0.2 that are left of the red bowl.
block_names = ['red block', 'blue block']
red_bowl_pos = get_pos('red bowl')
use_block_names = [name for name in block_names
                    if get_pos(name)[0] < red_bowl_pos[0]]
use_block_names = get_objs_bigger_than_area_th(use_block_names, 0.2)
ret_val = use_block_names

```

We describe more on prompt engineering in the Appendix A.

D. 语言模型程序作为策略

在机器人策略的背景下，语言模型程序可以根据自然语言指令组合感知到控制的反馈逻辑，其中感知模型的高层输出（状态）可以通过编程方式操作，并用于为低层控制应用程序编程接口（动作）的参数提供信息。关于可用感知和控制应用程序编程接口的先验信息可以通过示例和提示进行引导。这些应用程序编程接口将语言模型程序“接地”到真实世界的机器人系统，感知和控制算法的改进可以直接提升基于语言模型程序的策略能力。例如，在下面的真实世界实验中，我们使用最近开发的开放词汇目标检测模型，如现成的 ViLD[3] 和 MDETR[2]，来获取对象位置和边界框。基于语言模型程序的策略的好处有三方面：它们（i）可以将策略代码和参数适应于未见过的自然语言指令所指定的新任务和行为，（ii）可以通过引导开放词汇感知系统和/或显著性模型来泛化到新对象和环境，以及（iii）不需要任何额外的数据收集或模型训练。生成的计划和策略也是可解释的，因为它们以代码表示，便于修改和重用。将语言模型程序用于高层用户交互继承了大型语言模型的优势，包括解析具有常识知识的表达性自然语言、考虑先前的上下文、多语言能力以及参与对话。在接下来的实验部分中，我们展示了语言模型程序在不同机器人和不同任务上的多种实例，展示了该方法的灵活能力和易用性。

IV. 实验

本文实验目标有三：（i）评估在不同语言模型上使用分层代码生成的影响并分析泛化模式，（ii）在模拟语言指令操作任务中将代码即策略（Code as Policies, CaP）与基线方法进行比较，（iii）在不同机器人系统上演示 CaP 以展示其灵活性与易用性。更多实验见附录，例如生成反应式控制器以平衡倒立摆并执行末端执行器阻抗控制（附录 F）。附录还包含所有实验的提示与响应。复现这些实验的视频和 Colab 笔记本可在项目网站上找到。由于评估开放式任务存在困难且缺乏可比较的基线，使用 CaP 的机器人系统定量评估仅限于 IV-D 节中的一组受限模拟任务，而在 IV-B、IV-C 和 IV-E 节中，我们展示了系统支持的全部命令范围，未进行定量评估。

表 I：分层代码生成在 RoboCodeGen 中解决更多问题（以通过率% 计），并随缩放定律（模型参数量增加）而提升。

Method	GPT-3 [12]		Codex [1]	
	6.7B	175B	cushman	davinci
Flat	3	68	54	81
Hierarchical	5	84	57	95

表 II：分层代码生成在 HumanEval [1] 的通用编码问题上表现更好（通过率%）。Greedy 表示使用 temperature=0. 解码 LLM，P@N 评估 N 个样本的正确性，使用 temperature=0.8。

	Greedy	P@1	P@10	P@100
Flat	45.7	34.9	75.1	90.9
Hierarchical	53.0	39.8	80.6	95.7

A. 代码生成基准上的分层语言模型程序

我们在两个代码生成基准上评估代码生成方法：（i）机器人主题的 RoboCodeGen 和

（ii）HumanEval [1]，包含标准代码生成问题。

RoboCodeGen：我们引入了一个新的基准，包含 37 个函数生成问题，与之前的代码生成基准相比有几个关键区别：

（i）它以机器人学为主题，包含空间推理问题（例如，找到一组点中最近的点）、几何推理问题（例如，检查一个边界框是否包含在另一个边界框中）以及控制问题（例如，比例微分控制）；（ii）允许并鼓励使用第三方库（例如 NumPy）；（iii）提供的函数头没有文档字符串或显式类型提示，因此大语言模型需要推断并使用常见约定；（iv）允许使用尚未定义的函数，这些函数可以通过分层代码生成来创建。示例基准问题可在附录 E 中找到。我们在通过 OpenAI 应用程序编程接口访问的四个大语言模型上进行评估¹。与标准基准 [1] 一样，我们的评估度量是生成的代码通过人工编写的单元测试的百分比。见表 I。领域特定语言模型（Codex 模型）通常表现更好。在每个模型家族中，性能随着模型规模的增大而提高。分层方法在所有情况下都表现更好，表明允许大语言模型将复杂函数分解为分层部分并分别生成每个部分的代码是有益的。我们还分析了代码生成性能在 [23] 中提出的五种泛化类型上的变化。分层方法对生产力（Productivity）的帮助最大，即当新指令需要比示例中更长的代码或更多逻辑层时。然而，这些改进仅出现在两个 davinci 模型上，而不是 cushman 模型上，这表明在分层代码生成能够带来进一步改进之前，需要先达到一定的代码生成能力水平。更多结果见附录 E.2。在 **HumanEval[1]** 上的评估表明，分层代码生成不仅改进了策略代码，还改进了通用代码。见表 II。取得的数值高于近期工作 [1]、[11]、[58]。更多细节见附录 D。

B. CaP：通过生成航点绘制形状

在这个领域中，我们让一个真实的 UR5e 机器人手臂通过生成并跟随一系列二维航点来绘制各种形状。对于

¹ 两个文本模型：6.7B GPT-3 模型 [12] 和 175B InstructGPT[22]。两个 Codex 模型 [1]：cushman 和 davinci，经过训练以生成代码。davinci 更大且更好。Codex 模型的规模未公开。

表 III: 每个任务 50 次试验的任务族成功率。

Train/Test	Task Family	CLIPort [36]	NL Planner	CaP (ours)
SA SI	Long-Horizon	78.80	86.40	97.20
SA SI	Spatial-Geometric	97.33	N/A	89.30
UA SI	Long-Horizon	36.80	88.00	97.60
UA SI	Spatial-Geometric	0.00	N/A	73.33
UA UI	Long-Horizon	0.00	64.00	80.00
UA UI	Spatial-Geometric	0.01	N/A	62.00

感知方面, 语言模型程序 (LMP) 被赋予检测物体位置的应用程序编程接口 (API), 这些接口通过现成的开放词汇物体检测器 MDETR 实现 [2]。动作方面, 提供了一个末端执行器轨迹跟随 API。共有四个 LMP: (i) 解析用户命令、维护会话并调用动作 API, (ii) 从语言描述中解析物体名称, (iii) 从语言描述中解析路径点, (iv) 生成新函数。在机器人上成功执行未见语言命令的示例见图 2c。该系统能够引发空间推理, 根据语言命令绘制全新形状。更多示例展示了解析精确尺寸、操作先前形状以及多步命令的能力, 以及完整提示, 见附录 H。

C. CaP: 桌面操作的拾取与放置策略

桌面操作领域要求 UR5e 机器人手臂在桌面上拾取和放置各种塑料玩具物体。该手臂配备吸盘夹持器和内置英特尔实感 D435 相机。我们提供感知 API, 通过 MDETR 检测物体的存在、位置和边界框 [2]。我们还提供了一个脚本化原语, 用于拾取物体并将其放置在目标位置。提示与上一领域类似, 只是轨迹解析被替换为位置解析。在机器人上执行未见语言命令的示例见图 2 的 a 和 b 面板, 展示了推理物体描述和空间关系的能力。其他使用历史上下文 (例如“撤销那个”)、通过几何 (例如“最小的”) 和空间 (例如“最右边的”) 描述推理物体的命令见附录 I。

D. CaP: 桌面操作仿真评估

我们在来自 [16]、[18] 的模拟桌面操作环境中评估了代码即策略 (CaP)。该设置要求 UR5e 机械臂和 Robotiq 2F85 夹爪操作 10 个彩色方块和 10 个彩色碗。我们继承了全部 8 个任务, 因其多步骤特性而被称为“长时程”任务 (例如, “将方块放入匹配的碗中”)。我们定义了 6 个新任务, 这些任务需要更具挑战性和精确的空间几何推理能力 (例如, “将方块排成对角线”)。每个任务由一些属性参数化 (例如, “拿起<物体>并将其放置在<角落>”), 这些属性在每次试验中随机采样。我们将任务指令 (I) 和属性 (A) 划分为“已见” (SI, SA) 和“未见” (UI, UA) 类别, 其中“已见”意味着允许出现在提示中或被训练 (在监督基线的情况下)。更多细节见附录 K。我们考虑两种基线:

(i) 语言条件多任务 CLIPort[36] 策略, 通过模仿学习在 3 万次演示上训练, 以及 (ii) 使用自然语言而非代码的少样本提示大语言模型规划器。

结果见表 III。CaP 在具有已见属性和指令的任务上与监督 CLIPort 基线相比具有竞争力, 尽管仅对每个任务进行了一次示例回放的少样本提示。对于未见任务属性, CLIPort 的性能显著下降, 而基于大语言模型的方法保持相似性能。在未见任务和属性上, 像 CLIPort 这样的端到端系统难以泛化, 而 CaP 优于直接使用语言的大语言模型推理 (在 [20] 中也观察到)。此外, 自然语言规划器 [14]、[16]–[18] 不适用于需要精确数值空间几何推理的任务。我们还在附录 C 中展示了使用代码推理相对于自然语言 (直接问答和思维链 [47]) 的优势, 特别是前者执行精确数值计算的能力。

E. CaP: 移动机器人导航与操作

在该领域中, 配备移动底座和七自由度机械臂的机器人需在真实厨房环境中执行导航与操作任务。感知方面, 语言模型程序 (LMPs) 通过基于视觉语言检测器 (ViLD) 实现的目标检测应用程序编程接口获取信息 [3]。动作方面, 机器人可通过名称和坐标调用导航至指定位置及抓取物体的应用程序编程接口。未见语言指令在机器人上的执行示例见图 2。该领域表明, 代码作为策略 (CaP) 能够跨不同机器人系统及不同应用程序编程接口部署于现实任务中。此外, 它还展示了遵循包含控制结构的长时程反应式指令以及精确空间推理的能力, 这是先前工作难以实现的 [16], [17], [36]。提示及更多示例见附录 J。

五、讨论与局限性

CaP 在机器人堆栈的特定层级实现泛化: 解读自然语言指令, 处理感知输出, 然后参数化控制原语的低维输入。这适用于感知与控制解耦的系统, 并赋予一定程度的泛化能力 (源自预训练的大语言模型), 而无需端到端学习所需的大量数据收集。本文方法还继承了大语言模型与代码编写无关的能力, 例如支持非英语语言或表情符号的指令 (附录 L)。CaP 还能表达跨具身计划, 根据可用的应用程序编程接口 (API) 以不同方式执行相同任务 (附录 M)。然而, 这种能力在现有大语言模型上较为脆弱, 可能需要针对特定领域代码训练的更大模型。目前 CaP 受限于以下范围: (i) 感知 API 能够描述的内容 (例如, 至今没有视觉语言模型能描述轨迹是否“颠簸”或“更呈 C 形”), 以及 (ii) 可用的控制原语。只有少数命名的原语参数可以调整, 而不会使提示过饱和。CaP 也难以解读明显更长或更复杂的命令, 或在与给定示例不同的抽象层级上操作。在桌面操作领域, 语言模型程序 (LMP) 很难“用积木搭建房屋”, 因为没有关于构建复杂三维结构的示例。本文方法还假设所有给定指令都是可行的, 并且无法先验判断响应是否正确。

致谢

特别感谢维卡斯·辛德瓦尼 (Vikas Sindhwani) 和文森特·范霍克 (Vincent Vanhoucke) 对写作提出的宝贵反馈, 以及查德·布杜 (Chad Boodoo) 在运营和硬件支持方面的帮助。

参考文献

- [1] M. Chen, J. Twarek, H. Jun, Q. Yuan, H. P. d. O. Pinto, J. Kaplan, H. Edwards, Y. Burda, N. Joseph, G. Brockman *et al.*, "Evaluating large language models trained on code," *arXiv:2107.03374*, 2021.
- [2] A. Kamath, M. Singh, Y. LeCun, G. Synnaeve, I. Misra, and N. Carion, "Mdetr-modulated detection for end-to-end multi-modal understanding," in *ICCV*, 2021.
- [3] X. Gu, T.-Y. Lin, W. Kuo, and Y. Cui, "Open-vocabulary object detection via vision and language knowledge distillation," *arXiv:2104.13921*, 2021.
- [4] S. Tellex, N. Gopalan, H. Kress-Gazit, and C. Matuszek, "Robots that use language," *Review of Control, Robotics, and Autonomous Systems*, 2020.
- [5] T. Winograd, "Procedures as a representation for data in a computer program for understanding natural language," *MIT PROJECT MAC*, 1971.
- [6] J. Dzifcak, M. Scheutz, C. Baral, and P. Schermerhorn, "What to do and how to do it: Translating natural language directives into temporal and dynamic logic representation for goal management and action execution," in *ICRA*, 2009.
- [7] Y. Artzi and L. Zettlemoyer, "Weakly supervised learning of semantic parsers for mapping instructions to actions," *TACL*, 2013.
- [8] C. Lynch and P. Sermanet, "Language conditioned imitation learning over unstructured data," *arXiv:2005.07648*, 2020.
- [9] E. Jang, A. Irpan, M. Khansari, D. Kappler, F. Ebert, C. Lynch, S. Levine, and C. Finn, "Bc-z: Zero-shot task generalization with robotic imitation learning," in *CoRL*, 2022.
- [10] O. Mees, L. Hermann, E. Rosete-Beas, and W. Burgard, "Calvin: A benchmark for language-conditioned policy learning for long-horizon robot manipulation tasks," *RA-L*, 2022.
- [11] A. Chowdhery, S. Narang, J. Devlin, M. Bosma, G. Mishra, A. Roberts, P. Barham, H. W. Chung, C. Sutton, S. Gehrmann *et al.*, "Palm: Scaling language modeling with pathways," *arXiv:2204.02311*, 2022.
- [12] T. Brown, B. Mann, N. Ryder, M. Subbiah, J. D. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell *et al.*, "Language models are few-shot learners," *NeurIPS*, 2020.
- [13] S. Zhang, S. Roller, N. Goyal, M. Artetxe, M. Chen, S. Chen, C. Dewan, M. Diab, X. Li, X. V. Lin *et al.*, "Opt: Open pre-trained transformer language models," *arXiv:2205.01068*, 2022.
- [14] W. Huang, P. Abbeel, D. Pathak, and I. Mordatch, "Language models as zero-shot planners: Extracting actionable knowledge for embodied agents," *arXiv:2201.07207*, 2022.
- [15] T. Kojima, S. S. Gu, M. Reid, Y. Matsuo, and Y. Iwasawa, "Large language models are zero-shot reasoners," *arXiv:2205.11916*, 2022.
- [16] A. Zeng, A. Wong, S. Welker, K. Choromanski, F. Tombari, A. Purohit, M. Ryo, V. Sindhwani, J. Lee, V. Vanhoucke *et al.*, "Socratic models: Composing zero-shot multimodal reasoning with language," *arXiv:2204.00598*, 2022.
- [17] M. Ahn, A. Brohan, N. Brown, Y. Chebotar, O. Cortes, B. David, C. Finn, K. Gopalakrishnan, K. Hausman, A. Herzog *et al.*, "Do as i can, not as i say: Grounding language in robotic affordances," *arXiv:2204.01691*, 2022.
- [18] W. Huang, F. Xia, T. Xiao, H. Chan, J. Liang, P. Florence, A. Zeng, J. Tompson, I. Mordatch, Y. Chebotar, P. Sermanet, N. Brown, T. Jackson, L. Luu, S. Levine, K. Hausman, and B. Ichter, "Inner monologue: Embodied reasoning through planning with language models," in *arXiv:2207.05608*, 2022.
- [19] P. Florence, C. Lynch, A. Zeng, O. A. Ramirez, A. Wahid, L. Downs, A. Wong, J. Lee, I. Mordatch, and J. Tompson, "Implicit behavioral cloning," in *CoRL*, 2022.
- [20] A. Zeng, "Learning visual affordances for robotic manipulation," Ph.D. dissertation, Princeton University, 2019.
- [21] D. Kalashnikov, A. Irpan, P. Pastor, J. Ibarz, A. Herzog, E. Jang, D. Quillen, E. Holly, M. Kalakrishnan, V. Vanhoucke *et al.*, "Scalable deep reinforcement learning for vision-based robotic manipulation," in *CoRL*, 2018.
- [22] L. Ouyang, J. Wu, X. Jiang, D. Almeida, C. L. Wainwright, P. Mishkin, C. Zhang, S. Agarwal, K. Slama, A. Ray *et al.*, "Training language models to follow instructions with human feedback," *arXiv:2203.02155*, 2022.
- [23] D. Hupkes, V. Dankers, M. Mul, and E. Bruni, "Compositionality decomposed: How do neural networks generalise?" *JAIR*, 2020.
- [24] C. Breazeal, K. Dautenhahn, and T. Kanda, "Social robotics," *Springer handbook of robotics*, 2016.
- [25] T. Kollar, S. Tellex, D. Roy, and N. Roy, "Toward understanding natural language directions," in *HRI*, 2010.
- [26] J. Luketina, N. Nardelli, G. Farquhar, J. N. Foerster, J. Andreas, E. Grefenstette, S. Whiteson, and T. Rocktäschel, "A survey of reinforcement learning informed by natural language," in *IJCAI*, 2019.
- [27] M. MacMahon, B. Stankiewicz, and B. Kuipers, "Walk the talk: Connecting language, knowledge, and action in route instructions," *AAAI*, 2006.
- [28] J. Thomason, S. Zhang, R. J. Mooney, and P. Stone, "Learning to interpret natural language commands through human-robot dialog," in *IJCAI*, 2015.
- [29] S. Tellex, T. Kollar, S. Dickerson, M. Walter, A. Banerjee, S. Teller, and N. Roy, "Understanding natural language commands for robotic navigation and mobile manipulation," in *AAAI*, 2011.
- [30] D. Shah, B. Osinski, B. Ichter, and S. Levine, "Lm-nav: Robotic navigation with large pre-trained models of language, vision, and action," *arXiv:2207.04429*, 2022.
- [31] C. Matuszek, E. Herbst, L. Zettlemoyer, and D. Fox, "Learning to parse natural language commands to a robot control system," in *Experimental robotics*, 2013.
- [32] J. Thomason, A. Padmakumar, J. Sinapov, N. Walker, Y. Jiang, H. Yedidsion, J. Hart, P. Stone, and R. Mooney, "Jointly improving parsing and perception for natural language commands through human-robot dialog," *JAIR*, 2020.
- [33] S. Nair, E. Mitchell, K. Chen, S. Savarese, C. Finn *et al.*, "Learning language-conditioned robot behavior from offline data and crowd-sourced annotation," in *CoRL*, 2022.
- [34] J. Andreas, D. Klein, and S. Levine, "Learning with latent language," *arXiv:1711.00482*, 2017.
- [35] P. Sharma, B. Sundaralingam, V. Blukis, C. Paxton, T. Hermans, A. Torralba, J. Andreas, and D. Fox, "Correcting robot plans with natural language feedback," *arXiv:2204.05186*, 2022.
- [36] M. Shridhar, L. Manuelli, and D. Fox, "Cliport: What and where pathways for robotic manipulation," in *CoRL*, 2021.
- [37] S. Stepputtis, J. Campbell, M. Phielipp, S. Lee, C. Baral, and H. Ben Amor, "Language-conditioned imitation learning for robot manipulation tasks," *NeurIPS*, 2020.
- [38] Y. Jiang, S. S. Gu, K. P. Murphy, and C. Finn, "Language as an abstraction for hierarchical deep reinforcement learning," *NeurIPS*, 2019.
- [39] P. Goyal, S. Niekum, and R. J. Mooney, "Pixl2r: Guiding reinforcement learning using natural language by mapping pixels to rewards," *arXiv:2007.15543*, 2020.
- [40] G. Cideron, M. Seurin, F. Strub, and O. Pietquin, "Self-educated language agent with hindsight experience replay for instruction following," *DeepMind*, 2019.
- [41] D. Misra, J. Langford, and Y. Artzi, "Mapping instructions and visual observations to actions with reinforcement learning," *arXiv:1704.08795*, 2017.
- [42] A. Akakia, C. Colas, P.-Y. Oudeyer, M. Chetouani, and O. Sigaud, "Grounding language to autonomously-acquired skills via goal generation," *arXiv:2006.07185*, 2020.
- [43] I. Drori, S. Zhang, R. Shuttlesworth, L. Tang, A. Lu, E. Ke, K. Liu, L. Chen, S. Tran, N. Cheng *et al.*, "A neural network solves, explains, and generates university math problems by program synthesis and few-shot learning at human level," *PNAS*, 2022.
- [44] A. Lewkowycz, A. Andreassen, D. Dohan, E. Dyer, H. Michalewski, V. Ramasesh, A. Slone, C. Anil, I. Schlag, T. Gutman-Solo *et al.*, "Solving quantitative reasoning problems with language models," *arXiv:2206.14858*, 2022.
- [45] K. Cobbe, V. Kosaraju, M. Bavarian, J. Hilton, R. Nakano, C. Hesse, and J. Schulman, "Training verifiers to solve math word problems," *arXiv:2110.14168*, 2021.
- [46] D. Zhou, N. Schärli, L. Hou, J. Wei, N. Scales, X. Wang, D. Schuurmans, O. Bousquet, Q. Le, and E. Chi, "Least-to-most prompting enables complex reasoning in large language models," *arXiv:2205.10625*, 2022.
- [47] J. Wei, X. Wang, D. Schuurmans, M. Bosma, B. Ichter, F. Xia, E. Chi, Q. Le, and D. Zhou, "Chain of thought prompting elicits reasoning in large language models," *arXiv:2201.11903*, 2022.
- [48] J. Austin, A. Odena, M. Nye, M. Bosma, H. Michalewski, D. Dohan, E. Jiang, C. Cai, M. Terry, Q. Le *et al.*, "Program synthesis with large language models," *arXiv:2108.07732*, 2021.
- [49] K. Ellis, C. Wong, M. Nye, M. Sable-Meyer, L. Cary, L. Morales, L. Hewitt, A. Solar-Lezama, and J. B. Tenenbaum, "Dreamcoder: Growing generalizable, interpretable knowledge with wake-sleep bayesian program learning," *arXiv:2006.08381*, 2020.
- [50] L. Tian, K. Ellis, M. Kryven, and J. Tenenbaum, "Learning abstract structure for drawing by efficient motor program induction," *NeurIPS*, 2020.
- [51] D. Trivedi, J. Zhang, S.-H. Sun, and J. J. Lim, "Learning to synthesize programs as interpretable and generalizable policies," *NeurIPS*, 2021.
- [52] O. Mees and W. Burgard, "Composing pick-and-place tasks by grounding language," in *ISER*, 2020.
- [53] W. Liu, C. Paxton, T. Hermans, and D. Fox, "Structformer: Learning spatial structure for language-guided semantic rearrangement of novel objects," in *ICRA*, 2022.
- [54] W. Yuan, C. Paxton, K. Desingh, and D. Fox, "Sornet: Spatial object-centric representations for sequential manipulation," in *CoRL*, 2022.
- [55] A. Buckler, L. Figueredo, S. Haddadin, A. Kapoor, S. Ma, and R. Bonatti, "Reshaping robot trajectories using natural language commands: A study of multi-modal data alignment using transformers," *arXiv:2203.13411*, 2022.

- [56] A. Bobu, C. Paxton, W. Yang, B. Sundaralingam, Y.-W. Chao, M. Cakmak, and D. Fox, "Learning perceptual concepts by bootstrapping from human queries," *RA-L*, 2022.
- [57] J. Wu, L. Ouyang, D. M. Ziegler, N. Stiennon, R. Lowe, J. Leike, and P. Christiano, "Recursively summarizing books with human feedback," *arXiv:2109.10862*, 2021.
- [58] F. F. Xu, U. Alon, G. Neubig, and V. J. Hellendoorn, "A systematic evaluation of large language models of code," in *MAPS*, 2022.
- [59] K. Zakka, A. Zeng, P. Florence, J. Tompson, J. Bohg, and D. Dwibedi, "Xirl: Cross-embodiment inverse reinforcement learning," in *CoRL*. PMLR, 2022.
- [60] A. Ganapathi, P. Florence, J. Varley, K. Burns, K. Goldberg, and A. Zeng, "Implicit kinematic policies: Unifying joint and cartesian action spaces in end-to-end robot learning," *arXiv:2203.01983*, 2022.

附录

A. 提示工程

利用语言模型程序（LMP）通过代码生成可靠地完成任务需要细致的提示工程。虽然这些提示不必冗长，但必须相关且具体。本文讨论在开发提示时遵循的若干通用准则。提示中包含无缺陷的代码至关重要。提示中的缺陷会导致响应不可靠且不正确。反之，若语言模型程序针对给定指令编写了错误代码，提示工程师应首先核实提示本身，尤其是与指令最密切相关的示例，是否无缺陷。为减少与语法错误相关的缺陷，一种简单方法是在具有语法高亮功能的代码编辑器中编写提示。许多情况下，提示包含名称含糊的变量或函数。为此类条件下产生可靠响应，提示中的示例应一致地处理这些歧义。若名为点的变量在一个示例中被视为数值计算库（NumPy）数组对象，而在另一示例中被视为几何库（Shapely）点对象，语言模型程序将无法“决定”采用哪种约定，从而导致响应不可靠。处理歧义的另一方法是提供非正式类型提示，例如在变量名后附加 `_np` 以指示其类型，或在函数名后附加该标记以指示返回变量的类型。一般而言，更具体的变量和函数名能产生更一致的结果。对于第三方库的使用，提示中包含导入语句可能并非必需，因为我们发现语言模型程序无需这些语句即可生成调用数值计算库（NumPy）和科学计算库（SciPy）的代码。然而，显式导入语句确实能提高可靠性，并在需要时增加语言模型程序使用这些库的可能性。对于第一方库的使用，遵循流行命名规范的有意义函数名（例如以 `set_` 和 `get_` 开头）以及指定返回对象格式（例如 `get_bbox_xyxy`）能引导更准确的用法。提示中的导入语句应按照导入函数的格式书写。例如，在 Python 中这意味着使用从工具模块导入 `function_name`，而非导入 `function_name`。若采用后者，语言模型程序可能将导入的名称视为包，生成的代码可能写成

`function_name.function_name()`。一类语言模型程序失败与代码生成正确性相关。例如，调用内部或外部应用程序编程接口（API）时的细微编码错误（如缺少参数）可通过展示正确使用法的提示或示例修复。对变量类型的错误假设也可用类似方式修正。其他编码失败可通过描述性函数名鼓励适当的库使用（`perform_function_with_np0`）或简洁的代码逻辑（# 用一行实现）来解决。虽然可以使用大语言模型（LLM）编辑代码并修复缺陷（例如使用开放人工智能（OpenAI）的代码编辑应用程序编程接口），但根据我们的经验，这种方法产生的结果不一致（并非总能纠正错误，有时还会改变函数的功能），因此我们在实验中未采用此方法。

B. 方法部分提示

1) 基于语言的推理：完整提示：

```
objs = ['green block', 'green bowl', 'yellow block', 'yellow bowl']
# the yellow block.
ret_val = 'yellow block'
# the blocks.
ret_val = ['green block', 'yellow block']
```

2) *First-party*: Full prompt:

```
from utils import get_pos, put_first_on_second
objs = ['gray block', 'gray bowl']
# put the gray block on the gray bowl.
put_first_on_second('gray block', 'gray bowl')
objs = ['purple block', 'purple bowl']
# move the purple bowl toward the left.
target_pos = get_pos('purple bowl') + [-0.3, 0]
put_first_on_second('purple bowl', target_pos)
```

3) 结合语言推理、第三方库和第一方库：完整提示：

```
import numpy as np
from utils import get_pos, put_first_on_second
objs = ['cyan block', 'cyan bowl', 'pink bowl']
# put the cyan block in cyan bowl.
put_first_on_second('cyan block', 'cyan bowl')
objs = ['gray block', 'silver block', 'gray bowl']
# place the top most block on the gray bowl.
names = ['gray block', 'silver block']
positions = np.array([get_pos(name) for name in names])
name = names[np.argmax(positions[:,1])]
put_first_on_second(name, 'gray bowl')
objs = ['purple block', 'purple bowl']
# put the purple bowl to the left of the purple block.
target_pos = get_pos('purple block') + [-0.3, 0]
put_first_on_second('purple bowl', target_pos)
```

4) 语言模型程序（LMP）可组合：完整提示：

```
import numpy as np
from utils import get_pos, put_first_on_second, parse_obj
objs = ['yellow block', 'yellow bowl', 'gray block', 'gray bowl']
# move the sun colored block toward the left.
block_name = parse_obj('sun colored block')
target_pos = get_pos(block_name) + [-0.3, 0]
put_first_on_second(block_name, target_pos)
objs = ['white block', 'white bowl', 'yellow block', 'yellow bowl']
# place the block closest to the blue bowl on the other bowl.
block_name = parse_obj('the block closest to the blue bowl')
bowl_name = parse_obj('a bowl other than the blue bowl')
put_first_on_second(block_name, bowl_name)
```

5) *parse_obj prompt*.: Full prompt:

```
import numpy as np
from utils import get_pos
objs = ['brown bowl', 'green block', 'brown block', 'green bowl']
# the blocks.
ret_val = ['brown block', 'green block']
# the sky colored block.
ret_val = 'blue block'
objs = ['orange block', 'cyan block', 'purple bowl', 'gray bowl']
# the right most block.
block_names = ['orange block', 'cyan block']
block_positions = np.array([
    get_pos(block_name) for block_name in block_names])
right_block_name = block_names[np.argmax(block_positions[:, 0])]
ret_val = right_block_name
```

C. 使用代码推理与自然语言推理

为探究如何通过语言模型程序（LMP）而非自然语言来执行与机器人相关的推理，我们构建了一个基准，包含两组任务：（i）根据空间几何描述选择场景中的物体；（ii）根据空间几何描述选择位置坐标。物体选择包含 28 个问题，指令如“找出离蓝色碗最近的方块名称”，其中方块和碗的位置列表作为输入上下文在提示中提供。

位置选择包含 23 个问题，指令如“在从青色碗到蓝色碗的直线上插值 3 个点”。若语言模型生成的所有坐标与真实值相差在 1 厘米以内，则视为正确。我们将 LMP 与两种自然语言推理变体进行比较：（i）普通方法（**Vanilla**），给定场景描述（如物体位置列表）和问题，直接输出答案（例如，“问：最上面的方块是什么？”→“答：红色方块”）；（ii）思维链（CoT）[47]，在提示中给出中间步骤示例，逐步推理（例如，鼓励语言模型先列出场景中所有方块的 y 坐标，再识别最上面的方块）。

表四：使用代码进行空间几何推理的成功率（平均百分比）高于使用普通自然语言或思维链提示。

Tasks	Natural Language		Code
	Vanilla	CoT [47]	LMP (ours)
Object Selection	39	68	96
Position Selection	30	48	100
Total	35	58	98

表 IV 中的结果表明，语言模型规划器（LMPs）达到了 90% 以上的准确率，优于思维链（CoT），而思维链又优于基础模型（Vanilla）。思维链使大语言模型（LLMs）能够推理关系和顺序（例如，哪个坐标在另一个坐标的右侧），但在精确和多步数值计算上会出现失败。相比之下，来自语言模型规划器的代码可以使用 Python 执行此类计算，并且它们经常利用外部库来执行更复杂的操作（例如，使用 NumPy 进行向量加法）。思维链和语言模型规划器并非互斥——可以通过提示“逐步”代码生成来借助思维链解决更复杂的任务，但这是本文未探索的方向。

D. CodeGen HumanEval 附加结果

这里我们提供 HumanEval 实验的附加结果。总共测试了更大的 Codex 模型（code-davinci-002）的三种变体。我们的方法是分层代码生成（Hier. CodeGen）+ 分层提示（Hier Prompts），其中提示通过包含此类示例来鼓励大语言模型调用尚未定义的函数。为了比较，我们评估了扁平代码生成（Flat CodeGen）+ 无提示（No Prompt），即直接使用大语言模型，以及扁平代码生成 + 扁平提示（Flat Prompt），以便与扁平代码生成进行公平比较，因为我们的分层方法包含提示。提示仅包含 2 个示例：扁平代码生成的提示：

```
prompt_f_gen_flat = ""
def get_total(xs: List[float]) -> float:
    """Find the sum of a list of numbers called xs.
    """
    return sum(xs)
# end of function

def get_abs_diff_between_means(xs0: List[float],
                               xs1: List[float]) -> float:
    """Get the absolute difference between the means of two
    lists of numbers.
    m0 = sum(xs0) / len(xs0)
    m1 = sum(xs1) / len(xs1)
    return abs(m0 - m1) # end of function
```

分层代码生成的提示：

```
def get_total(xs: List[float]) -> float:
    """Find the sum of a list of numbers called xs.
    """
    return sum(xs)
# end of function

def get_abs_diff_between_means(xs0: List[float],
                               xs1: List[float]) -> float:
    """Get the absolute difference between the means of two lists of
    numbers.
    """
    m0 = get_mean(xs0)
    m1 = get_mean(xs1)
    return abs(m0 - m1)
# end of function
```

注意，分层提示的唯一区别是使用尚未定义的函数 `get_mean`，而不是直接计算均值。这“允许”大语言模型生成也调用尚未定义的函数的代码。我们报告使用最可能输出（“贪婪”，通过将温度设为 0 实现）时的通过率，以及在温度设为 0.8 时从不同采样数量（1、10 和 100）的解决方案中至少有一个解决方案的通过率，与先前工作 [1]、[11]、[58] 中使用的类似。

表 V：分层代码生成在标准 HumanEval 基准 [1] 的通用编码问题上也表现更好（以 % 通过率计）。对于列，Greedy 表示使用 temperature=0 解码大语言模型，而 P@N 表示评估从使用 temperature=0.8 解码的大语言模型中采样的 N 个样本的正确性。

	Greedy	P@1	P@10	P@100
code-davinci-001 [11]	-	36.0	-	81.7
PaLM Coder [11]	-	36.0	-	88.4
Flat CodeGen + No Prompt	45.7	34.9	75.1	90.9
Flat CodeGen + Flat Prompts	50.6	36.6	77.6	93.3
Hier. CodeGen + Hier Prompts	53.0	39.8	80.6	95.7

结果见表二。在所有实例中，分层代码生成均优于扁平代码生成，且所达到的数值高于近期工作 [1]、[11]、[58] 中报告的结果。注意，本文使用代码达芬奇 002（code-davinci-002），而先前工作使用代码达芬奇 001（code-davinci-001），但分层带来的相对改进在各处保持一致。在人类评估（HumanEval）的 164 个问题中，6.5% 的问题触发了分层代码生成，其中扁平代码生成（Flat CodeGen）两种变体的成功率为 44%，而分层代码生成（Hier CodeGen）的成功率为 56%。虽然采样 100 个响应时的成功率在各处均超过 90%，但本文指出，对语言模型规划器（LMP）而言，采样多个解并不实用，因为其需要以零样本方式执行任务，且无需事先设计单元测试。因此，对于语言模型规划器，本文始终将温度设为 0 并使用最可能的输出。

E. 机器人代码生成基准

1) 示例问题：以下是四类基准问题及其示例：• 使用 NumPy 进行向量运算：点 = interpolate_pts_np(start, 结束, n) • 简单控制：u = pd_control(x_curr, x_goal, x_dot, Kp, Kv) • 使用 Shapely 操作形状：圆 = make_circle(radius, 中心) • 使用第一方库：

```
ret_val = obj_shape_does_not_contain_others(obj_name,
other_obj_names)
```

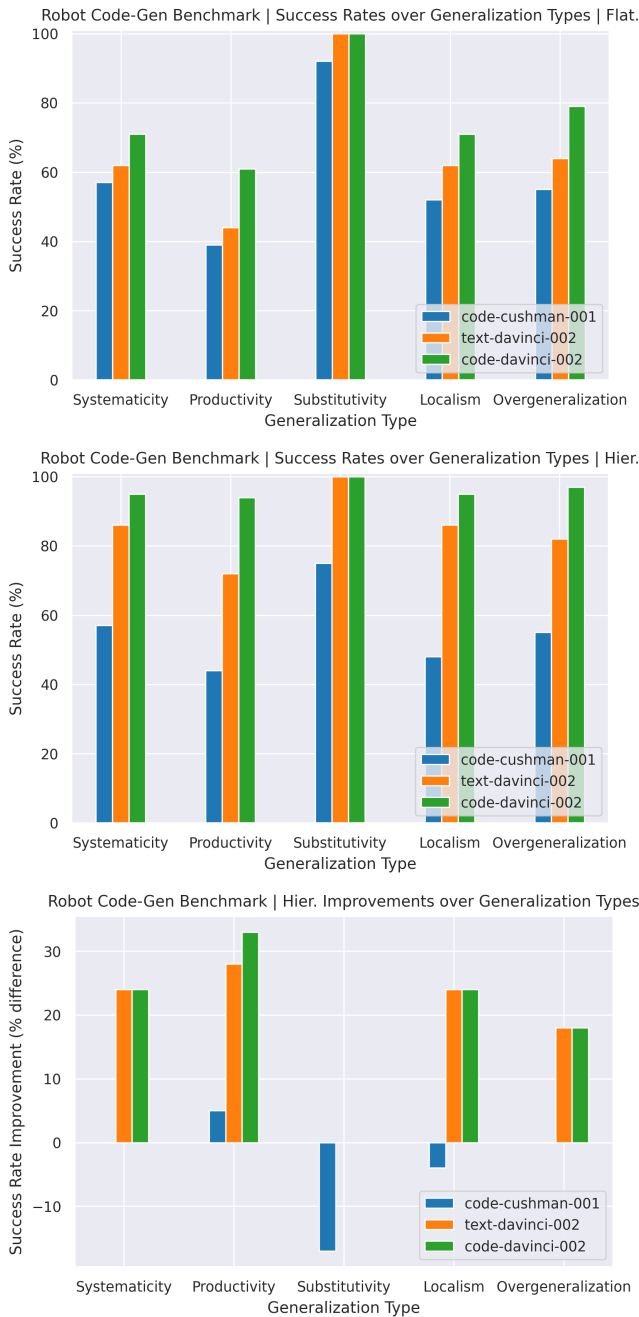


图 4：机器人代码生成基准在扁平（上）与分层（中）代码生成下的泛化类型性能，以及分层代码生成带来的性能提升（下）。

对于最后一类，本文在提示中以提示（Hints）形式提供第一方函数的导入，例如按名称获取物体几何信息的函数。2）泛化分析：本文分析代码生成在 [23] 中描述的五类泛化上的表现，其中泛化通过比较提示中给出的示例与基准中给出的新指令来评估。本文给出应用于该基准的五类泛化的描述。具体而言，本文认为

若基准中某问题的指令或解满足以下条件，则解决该问题即展示了特定类型的泛化：

- 系统性：重组示例指令或代码片段的组成部分。
- 产出性：代码更长或包含比示例更多的层级（例如分层函数调用）。
- 可替换性：使用示例中的同义词或替换相似类别的词。
- 局部性：将已见的部分或概念用于不同目的。

• 过度泛化：使用新的应用程序编程接口调用或编程示例中未见的语言特征。图 4 报告了三种模型的结果。上图展示了使用扁平提示和扁平代码生成的结果，而下图则采用分层提示和分层代码生成。三种模型的相对排名在所有泛化类型中保持一致，其中代码达芬奇-002（code-davinci-002）表现最佳。最小的模型代码库什曼-001（code-cushman-001）在生产力和局部性方面表现尤其差，但在可替换性方面表现好得多。实际上，所有模型在可替换性上的高表现可能表明，语言模型在替换具有相似含义或来自相似类别的词语时特别稳健，而泛化到更复杂的命令（通过生产力实现更长答案或通过局部性维持局部结构）则更具挑战性。分层代码生成也为两种达芬奇模型带来了显著的性能提升，尤其是在生产力方面，使这些模型能够编写更长、更多层次的函数。然而，这些改进在库什曼模型上并不明显，其可替换性和局部性成功率实际上更低，这表明在分层代码生成能够带来进一步改进之前，需要达到一定的代码生成能力水平。

F. CaP：用于玩具任务的反应式控制器

语言模型程序（LMP）可以表达反应式控制器，我们在以下任务中展示了这一点：（i）OpenAI Gym 的 CartPole 平衡任务和（ii）末端执行器位置阻抗控制。使用相同的提示结构，两种控制器都可以零样本生成，这反映了语言模型中已存储的知识，当目标在函数签名中得到充分指定时，这些知识可以被调用：

```
# define function: direction = keep_pole_upright_with_pd_control(x,
x_dot, theta, theta_dot).
# info: direction is 1 if going right, 0 if going left.
def keep_pole_upright_with_pd_control(x, x_dot, theta, theta_dot):
    # define constants.
    kp = 1
    kd = 1
    # define direction.
    direction = 1
    # define error.
    error = theta
    # define error_dot.
    error_dot = theta_dot
    # define control.
    control = kp * error + kd * error_dot
    # define direction.
    if control < 0:
        direction = 0
    return direction
```

生成的语言模型程序表达了一个控制器，该控制器以小车位置、杆的角度及其速度作为输入，然后通过向下方的小车施加离散的向左

或向右的动作来平衡杆。这个控制器只是一个简单的控制器，需要初始化接近直立状态，但它可以稳定一个不稳定的不动点。

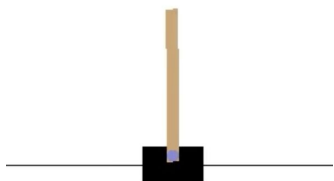


图 5: 语言模型程序可以平衡倒立摆

语言模型程序同样可以被提示来表达阻抗控制:

```
# define function: tau = ee_impedance_control(x_curr, x_goal,
                                             x_dot, K_x_mat, D_x_mat, J).
def ee_impedance_control(x_curr, x_goal, x_dot, K_x_mat,
                          D_x_mat, J):
    x_err = x_goal - x_curr
    x_dot_err = -x_dot
    tau = np.matmul(J.T,
                    np.matmul(K_x_mat, x_err) + np.matmul(D_x_mat, x_dot_err))
    return tau
```

利用关节力矩将机器人手臂末端执行器移向目标位置。该控制器功能上能够在 PyBullet 中控制 UR5e 机器人，但简化之处在于未补偿科里奥利力或重力。注意需要在函数签名中包含关于期望方向输出的额外信息以及使用比例微分控制的提示。没有这些提示，生成的函数可能看起来仍然合理（例如可能输出连续控制值而非离散值），但无法适用于该特定环境应用程序编程接口。对于输入增益的名称，`_mat` 是语言模型程序将其视为矩阵而非标量所必需的，`_x` 用于表明这些增益针对末端执行器而非关节。我们通过在 PyBullet 中指令 UR5e 机器人的末端执行器三维位置来演示该控制器的使用。默认比例微分增益 1 在该领域也能工作，无需额外调参，因为 CartPole 环境相对简单。更复杂的连续控制任务可能需要根据执行反馈实际调整增益，而我们的方法目前尚不支持。两个示例均表明可以生成简单的反应式控制器，但表达更复杂的控制器仍需更多工作。

G. 视觉语言模型

对于真实世界实验，我们使用现成的开放词汇目标检测模型 ViLD [3] 和 MDETR [2] 来执行目标检测、定位和分割。这些被称为视觉语言模型，因为它们以图像的自然语言描述（图注）作为输入，并尝试找到该描述中的对象。ViLD 用于移动机器人领域，而 MDETR 用于桌面操作和白板绘图领域。两种模型均在图像中给出轴对齐的边界框以及检测到的对象的逐像素分割掩码。为了将这些检测结果转换为

对于感知和动作（例如，脚本化的抓取原语）的三维坐标，我们从深度相机反投影相应的像素，该相机到机器人坐标系的变换已预先注册。当今视觉语言模型的鲁棒性仍有待提高，许多现实世界的失败可归因于检测不准确。此外，视觉语言模型还需要一定程度的提示工程。例如，MDETR 使用“正方形”一词比“方块”更可靠地检测方块，将我们的方法应用于新领域将需要对视觉语言模型进行一些提示工程。

H. 白板绘图

在该领域中，UR5e 机器人被要求在白板上绘制和擦除由自然语言描述的各种形状。干擦马克笔刚性附着在机器人末端执行器上。白板的尺寸、位置以及擦除器的位置是已知的。场景中可以增加其他物体以供命令引用（例如，在蓝色方块周围画一个圆）。在我们的演示中，我们使用谷歌云语音转文本和文本转语音应用程序编程接口，允许用户通过语音命令与系统交互，并听取机器人对命令的响应。提示。

- `draw_ui`: 用于解析用户命令和调用其他函数的高级用户界面 https://code-as-policies.github.io/prompts/draw_ui.txt
- `parse_obj_name`: 从自然语言描述中返回物体名称 https://codeas-policies.github.io/prompts/parse_obj_name.txt
- `parse_shape_pts`: 从自然语言描述中返回形状的二维路径点序列 https://codeas-policies.github.io/prompts/parse_shape_pts.txt
- `transform_shape_pts`: 对来自自然语言描述的二维点序列执行二维变换 https://codeas-policies.github.io/prompts/transform_shape_pts.txt
- `function_generation`: 从注释中定义函数 https://code-as-policies.github.io/prompts/fgen_simple.txt 应用程序编程接口。
- `get_obj_names()` - 获取场景中可用物体的列表。这些是预先指定的。
- `get_obj_pos(name)` - 按名称获取物体中心的二维位置。
- `draw(pts_2d)` - 通过命令机器人末端执行器在白板上跟随一系列点来绘制形状。

机器人首先移动到轨迹中第一个点上方的一点，向下移动直到检测到与白板接触，然后继续跟随轨迹的其余部分。

- `erase(pts_2d)` - 通过命令机器人擦除形状
末端执行器首先与擦除器建立接触（擦除器位置是硬编码的），然后跟随轨迹的其余部分。

指令。这些指令从一块空白白板开始，按顺序依次下达给机器人。完整视频和生成的代码见网站。

- 1) 在中间画一个 5 厘米的六边形
- 2) 画一条平分六边形的线
- 3) 把它们都放大
- 4) 擦掉六边形和线
- 5) 在右上角画一个圆作为太阳
- 6) 在底部画一条线作为地面
- 7) 在地面上画一个三角形作为金字塔
- 8) 在稍左位置绘制一个较小的金字塔
- 9) 在积木周围绘制圆圈
- 10) 在较甜的水果周围绘制一个正方形

I. 真实世界桌面操作

在该领域中，一台 UR5e 机器人被要求根据自然语言指令操作桌面上的物体。该机器人配备有吸盘夹爪，并且只能执行由二维俯视拾取和放置位置参数化的拾取与放置动作。机器人还需要通过使用提供的感知应用程序编程接口（API）来回答案关于场景的问题（例如，有多少个积木？）。在我们的演示中，我们使用谷歌云（Google Cloud）的语音转文本和文本转语音应用程序编程接口，使用户能够通过语音命令与系统交互，并听到机器人对命令和问题的回应。目前，提示仅支持一组唯一物体。这不是 CaP 的局限性，而是感知系统的局限性——我们没有一种好的方法在视觉语言模型（VLM）检测中持久化重复物体的身份。一个更复杂的跟踪感知世界状态的系统可以解决这个问题。提示。

• `tabletop_ui`: 用于解析用户命令和调用其他函数的高级用户界面
https://code-as-policies.github.io/prompts/tabletop_ui.txt • `parse_obj_name`: 从自然语言描述中返回物体名称

• `parse_position`: 从自然语言描述中返回二维位置
https://code-as-policies.github.io/prompts/parse_position.txt • `parse_question`: 返回对自然语言问题的响应（可以是数字、布尔值或字符串）

• `function_generation`: 从注释中定义函数
<https://code-as-policies.github.io/prompts/fgen.txt>

应用程序编程接口。

• `get_obj_names()` - 获取场景中可用物体的列表。这些是预先指定的。

• `get_obj_pos(name)` - 按名称获取物体中心的二维位置。

• `is_obj_visible(name)` - 按名称检查物体是否可见。

• `get_bbox(name)` - 按名称获取物体的二维轴对齐边界框。这是在机器人基坐标系中，而不是以像素为单位。

• `get_segmask(name)` - 按名称获取目标检测的分割掩码。这是以像素为单位的。

• `get_color_rgb(name)` - 按名称获取目标检测裁剪区域的平均 RGB 颜色。

• `get_corner_name(pos_2d)` - 获取最接近二维点的角点名称（例如右上角）。

• `get_side_name(pos_2d)` - 获取最接近二维点的边名称（例如左侧）。

• `denormalize_xy(normalized_pos_2d)` 将归一化二维坐标（每个值在 0 到 1 之间）转换为机器人坐标系中的实际二维坐标。

• `put_first_on_second(obj_name, 目标)` - 按名称拾取第一个物体，并将其放置在按名称指定的目标上。目标可以是另一个物体名称或二维位置。拾取和放置通过将吸盘直接移动到期望位置上方，向下移动直到检测到接触，然后接合或分离吸盘来完成。

• `say(message)` - 使用机器人扬声器播报消息。我们在桌面操作设置中展示了 CaP 在三个领域上的表现。每个领域的指令如下，并按顺序执行。完整视频和生成的代码见网站。

4 个积木领域的指令。1) 将积木在顶部附近排成一条水平线 2) 将天蓝色积木移到红色积木和从左数第二个积木之间 3) 你为什么移动绿色积木？ 4) 你移动了哪个积木？ 5) 将积木在中间周围排成一个正方形 6) 将正方形变大 7) 撤销该操作 8) 将正方形旋转 45 度 9) 你能扔积木吗？ 10) 将红色积木向下移动 5 厘米 11) 对其他积木执行相同操作 12) 从右上角开始，按顺时针方向将积木放在不同的角上 3 个积木和 3 个碗领域的指令。

1) 将红色积木放在最右边碗的左侧 2) 现在将其移到离它最远的一侧 3) 红色积木左侧有多少个碗？ 4) 将积木放入颜色不匹配的碗中 5) 将积木排成一条 20 厘米长的垂直线，位于蓝色碗下方 10 厘米处 6) 想象碗代表火山、森林和海洋 7) 同时想象积木是建筑物的组成部分 8) 现在在森林中建造一座塔 9) 展示当火山在海洋上喷发时会发生什么 水果、瓶子和盘子领域的指令。

1) 有多少个水果？ 2) 告诉我它们的名字 3) 绿色盘子上有水果吗？ 4) 将所有水果移到绿色盘子，将瓶子移到蓝色盘子 5) 将最小的水果移回黄色盘子 6) 等待直到你看到一个鸡蛋，然后将其放在绿色盘子上 7) 将最暗的物体放在有苹果的盘子里

的盘子里

J. 移动机器人

移动操作实验设置中，来自日常机器人（Everyday Robots）的机器人在真实世界办公室厨房中导航并与物体交互。该机器人具有移动底座和七自由度（7DoF）机械臂。为实现感知应用程序编程接口（API），我们主要使用机器人上的红绿蓝深度（RGBD）相机传感器。机器人如图 6 所示。

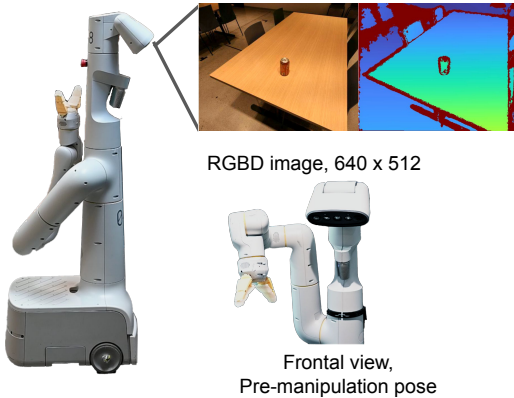


图 6：使用日常机器人（Everyday Robots）机器人进行移动操作的实验设置。

提示词。

- `mobile_ui`: 用于解析用户命令并调用其他函数的高级用户界面（UI）

https://code-as-policies.github.io/prompts/mobile_ui.txt •

`parse_obj_name`: 从自然语言描述中返回物体名称
https://codeas-policies.github.io/prompts/mobile_parse_obj_name.txt •

`parse_position`: 从自然语言描述中返回二维位置
https://codeas-policies.github.io/prompts/mobile_parse_pos.txt • `transform_traj`: 对来自自然语言描述的二维点序列执行二维变换

https://codeas-policies.github.io/prompts/mobile_transform_traj.txt • `function_generation`: 根据注释定义函数
https://code-as-policies.github.io/prompts/fgen_simple.txt 应用程序编程接口（API）。

- `get_obj_names()` - 获取场景中可用物体列表。这些是预先指定的。
- `get_obj_pos(name)` - 按名称获取物体中心的二维位置。
- `is_obj_visible(name)` - 返回机器人是否按名称看到某个物体。
- `get_visible_obj_names()` - 返回当前可见物体名称列表。
- `get_loc_names()` - 返回机器人可以导航到的所有预定义位置名称列表。
- `get_obj_pos(name)` - 按名称获取物体的三维位置。该物体必须当前可见。
- `get_loc_pos(name)` - 获取预定义位置的二维位置和一维角度。
- `get_robot_pos_and_angle` - 获取当前机器人三维位置和一维角度（朝向）。

- `goto_pos(pos_3d)` - 通过运行机器人内部运动规划器导航至三维位置。
- `goto_loc(name)` - 通过名称导航至某个位置，同样运行机器人内部运动规划器。
- `pick_obj(name)` - 按名称抓取物体。该物体必须当前可见。此功能通过使用 ViLD 目标检测的脚本化抓取原语实现。
- `place_at_pos(pos_3d)` - 将当前持有的物体放置到指定位置。
- `place_at_obj(name)` - 将当前持有的物体按名称放置到另一个物体之上。
- `say(message)` - 使用机器人扬声器播报消息。下面列出在移动机器人平台上执行的命令。前几项为导航相关任务，后几项为操作相关任务。对于后者的操作命令，请注意 CaP 通过在 Python 执行作用域中显式记录变量（本例中为机器人过去的位置）并在之后引用它们来形成“短期记忆”的能力。请参阅网站上的视频和生成代码。移动导航指令。1) 围绕办公椅在 3 米×2 米的矩形内移动 2) 再次执行该操作，但顺时针旋转 45 度 3) 围绕吧凳在 1.5 米的正方形内移动，根据需要重复多次，每步检查是否有香蕉，仅在看到香蕉时停止移动 4) 沿包含椅子的凸包移动 5) 在桌子和台面之间往返移动 3 次 移动操作指令。1) 桌子上有多少零食？ 2) 从书桌上拿起水瓶，将其放在桌子上水果的中间 3) 这是堆肥箱 4) 这是回收箱 5) 这是垃圾填埋箱 6) 可乐罐和苹果在桌子上 7) 将可乐罐和苹果放到对应的垃圾桶中

K. 仿真桌面操作评估

与真实世界桌面领域类似，我们构建了一个仿真桌面环境，其中配备 Robotiq 2F85 夹爪的 UR5e 机器人接收自然语言指令以完成重排任务。物体包括 10 个不同颜色的方块和 10 个不同颜色的碗。所提出的 CaP 通过脚本化物体检测器获得访问当前物体列表及其位置的应用程序编程接口，以及由坐标或物体名称参数化的抓取-放置运动原语。提示。

- `tabletop_ui`: 用于解析用户命令和调用其他函数的高层用户界面

https://codeas-policies.github.io/prompts/sim_tabletop_ui.txt

- `parse_obj_name`: 根据自然语言描述返回物体名称
https://codeas-policies.github.io/prompts/sim_parse_obj_name.txt
- `parse_position`: 根据自然语言描述返回二维位置
https://codeas-policies.github.io/prompts/sim_parse_position.txt
- `function_generation`: 根据注释定义函数
<https://code-as-policies.github.io/prompts/fgen.txt> 应用程序编程接口。
- `get_obj_names()` - 获取场景中可用物体列表。这些物体是预先指定的。
- `get_obj_pos(name)` - 按名称获取物体中心的二维位置。
- `denormalize_xy(normalized_pos_2d)` 将归一化二维坐标（每个值在 0 到 1 之间）转换为机器人坐标系中的实际二维坐标。
- `put_first_on_second(obj_name, 目标)` - 按名称拾取第一个物体，并将其放置在按名称指定的目标上。目标可以是另一个物体名称或二维位置。拾取和放置通过将吸盘式夹爪直接移动到期望位置上方，向下移动直到检测到接触，然后接合或脱离吸盘来完成。我们在以下任务上评估 CaP 和基线方法，其中每个任务对应一个唯一的指令模板（例如，“拿起<方块>并将其放在距离<碗><距离>的角落”），这些模板由特定属性（例如，<方块>）参数化。我们将任务分为指令和属性，并归入“已见”和“未见”类别，其中“已见”指令或属性允许出现在提示中或用于训练（对于有监督基线方法）。完整列表如下所示。注意，我们进一步将指令分为“长时程”和“空间几何”任务族。“长时程”指令为已见指令中的 1-5 和未见指令中的 1-3。“空间几何”指令为已见指令中的 5-8 和未见指令中的 4-6。

已见指令。

- 1) 拿起<方块 1>并将其放在（<方块 2>或<碗>）上
- 2) 堆叠所有方块
- 3) 将所有方块放在<角落/边>
- 4) 将方块放入<碗>中
- 5) 将所有方块放入颜色匹配的碗中
- 6) 拿起<碗><方向>的方块并将其放在<角落/边>
- 7) 拿起距离<碗><距离>的方块并将其放在<角落/边>
- 8) 从<方向>拿起第<n>个方块并将其放在<角落/边>

未见指令。

- 1) 将所有方块放在不同的角落
- 2) 将方块放入颜色不匹配的碗中
- 3) 将所有方块堆叠在<角落/边>
- 4) 拿起<方块 1>并将其放在<碗>的<方向><幅度>处

- 5) 拿起<积木 1>并将其放置在距离<碗>最近的<角落>处
- 6) 将所有积木排成一条<直线>线 已见属性。

1) <积木>: 蓝色积木、红色积木、绿色积木、橙色积木、黄色积木

2) <碗>: 蓝色碗、红色碗、绿色碗、橙色碗、黄色碗

3) <角落/边>: 左侧、左上角、顶边、右上角

4) <方向>: 上、左

5) <距离>: 最近

6) <幅度>: 一点

7) <序数>: 第一、第二

8) <直线>: 水平、垂直 未见属性。

1) <积木>: 粉色积木、青色积木、棕色积木、灰色积木、紫色积木

2) <碗>: 粉色碗、青色碗、棕色碗、灰色碗、紫色碗

3) <角落/边>: 右下角、底边、左下角

4) <方向>: 下、右

5) <距离>: 最远

6) <幅度>: 很多

7) <序数>: 第三、第四

8) <直线>: 对角线

在表六中，我们提供了详细的仿真结果，报告了细粒度任务类别的任务成功率。属性指<>字段，其值可被方法所见（例如，CLIPort 的训练集，基于语言的方法的提示）。指令指每行给出的模板化指令类型，也可以是已见或未见的。每个任务共进行 50 次试验，每次试验均采样属性和初始场景配置（积木和碗的类型、数量及位置）。注意，CLIPort 本身（无预言机）只是一个反馈策略，它不知道何时停止——在这种情况下，我们运行 CLIPort 策略的 10 个动作，并在结束时评估成功率。为了提高 CLIPort 的性能，我们使用一种变体，利用仿真中的预言机信息在检测到成功时停止策略（预言机终止）。

L. 大语言模型的附加能力

使用大型预训练大语言模型还意味着我们可以利用其超越代码编写的能力。例如，代码即策略可以解析非英语语言的命令以及表情符号。参见图 7。

M. 跨具身示例

代码即策略通过根据动作应用程序编程接口的不同以不同方式执行相同任务，展现出一定程度的跨具身支持 [59]、[60]。在下面的示例中，我们给出动作应用程序编程接口的提示，结果计划会根据机器人是全向还是单向而改变。我们注意到，这种能力在现有大语言模型中较为脆弱，无法可靠地适应差异很大的应用程序编程接口。更强的鲁棒性可能需要针对特定领域代码训练的更大模型。

表 6：不同任务场景下仿真桌面操作成功率的数据（%）。

	CLIPort（有预言终止）	CLIPort（无预言）	自然语言规划器	上下文感知规划（本文方法）
Seen Attributes, Seen Instructions				
Pick up the <object1> and place it on the (<object2> or <recepticle-bowl>)	88	44	98	100
Stack all the blocks	98	4	94	94
Put all the blocks on the <corner/side>	96	8	46	92
Put the blocks in the <recepticle-bowl>	100	22	94	100
Put all the blocks in the bowls with matching colors	12	14	100	100
Pick up the block to the <direction> of the <recepticle-bowl> and place it on the <corner/side>	100	80	N/A	72
Pick up the block <distance> to the <recepticle-bowl> and place it on the <corner/side>	92	54	N/A	98
Pick up the <nth> block from the <direction> and place it on the <corner/side>	100	38	N/A	98
Total	85.8	33.0	86.4	94.3
Long-Horizon Total	78.8	18.4	86.4	97.2
Spatial-Geometric Total	97.3	57.3	N/A	89.3
Unseen Attributes, Seen Instructions				
Pick up the <object1> and place it on the (<object2> or <recepticle-bowl>)	12	10	98	100
Stack all the blocks	96	8	96	100
Put all the blocks on the <corner/side>	0	0	58	100
Put the blocks in the <recepticle-bowl>	46	0	88	96
Put all the blocks in the bowls with matching colors	30	26	100	92
Pick up the block to the <direction> of the <recepticle-bowl> and place it on the <corner/side>	0	0	N/A	60
Pick up the block <distance> to the <recepticle-bowl> and place it on the <corner/side>	0	0	N/A	100
Pick up the <nth> block from the <direction> and place it on the <corner/side>	0	0	N/A	60
Total	23.0	5.5	88.0	88.5
Long-Horizon Total	36.8	8.8	88.0	97.6
Spatial-Geometric total	0.0	0.0	N/A	73.3
Unseen Attributes, Unseen Instructions				
Put all the blocks in different corners	0	0	60	98
Put the blocks in the bowls with mismatched colors	0	0	92	60
Stack all the blocks on the <corner/side>	0	0	40	82
Pick up the <object1> and place it <magnitude> to the <direction> of the <recepticle-bowl>	0	0	N/A	38
Pick up the <object1> and place it in the corner <distance> to the <recepticle-bowl>	4	0	N/A	58
Put all the blocks in a <line>	0	0	N/A	90
Total	0.7	0.0	64.0	71.0
Long-Horizon Total	0.0	0.0	64.0	80.0
Spatial-Geometric Total	1.3	0.0	N/A	62.0

```
import numpy as np
from utils import get_pos, put_first_on_second
objects = ['cyan block', 'cyan bowl', 'pink bowl']
# put the cyan block in cyan bowl.
put_first_on_second('cyan block', 'cyan bowl')
objects = ['gray block', 'silver block', 'gray bowl']
# place the top most block on the gray bowl.
names = ['gray block', 'silver block']
positions = np.array([get_pos(name) for name in names])
name = names[np.argmax(positions[:,1])]
put_first_on_second(name, 'gray bowl')
objects = ['purple block', 'purple bowl']
# put the purple bowl to the left of the purple block.
target_pos = get_pos('purple block') + [-0.3, 0]
put_first_on_second('purple bowl', target_pos)
objects = ['red block', 'red bowl', 'blue block', 'blue bowl']
# 把蓝色的木块放在红色的碗里。
put_first_on_second('blue block', 'red bowl')
objects = ['green block', 'cyan block']
# move the 10cm to the 🏠.
target_pos = get_pos('green block') + [-0.1, 0]
put_first_on_second('green block', target_pos)
```

```
# omnidirectional robot.
# available actions: move_up(dist), move_right(dist), move_back(dist).
# make a U shape 5 meters wide and 10 meters long.
robot.move_back(dist=10)
robot.move_right(dist=5)
robot.move_up(dist=10)
# unidirectional robot.
# available actions: turn_left(deg), move_forward(dist).
# make a U shape 3 meters wide and 8 meters long.
robot.move_forward(dist=8)
robot.turn_left(deg=90)
robot.move_forward(dist=3)
robot.turn_left(deg=90)
robot.move_forward(dist=8)
```

图 7：语言模型规划器继承了大型语言模型的优势，例如解析非英语语言和表情符号的命令。